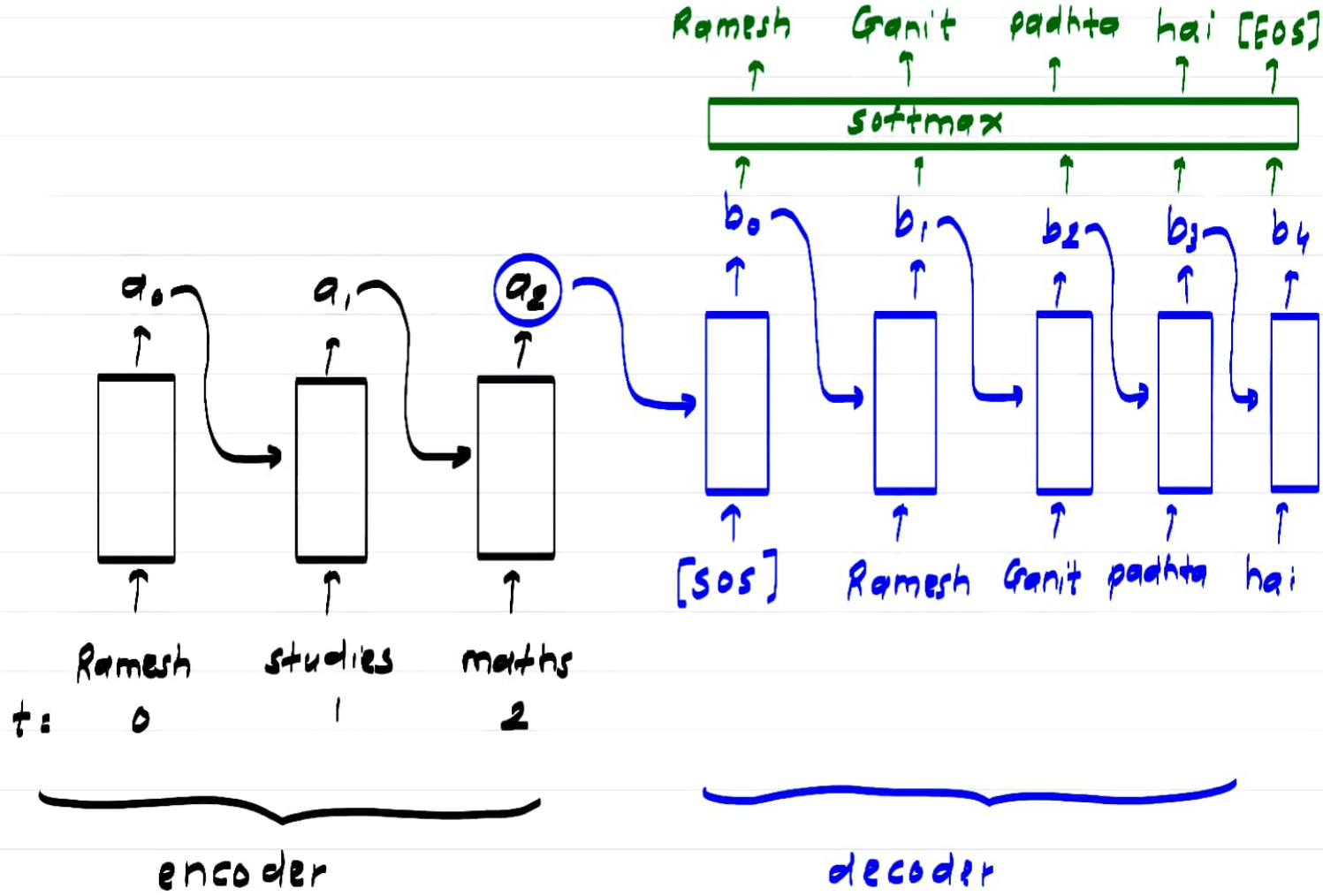
For seq. seq task we use encoder-decoder architecture

Encoder- Decoder Architecture:



Operations inside encoder block:

Lets assume we are using simple RNN in encoder block

$t=0$ $\quad z_0 = w_x x_0 + w_a a_{-1} + b$

$\quad a_0 = f(z_0)$

$t=1$ $\quad z_1 = w_x x_1 + w_a a_0 + b$

$\quad a_1 = f(z_1)$

$t=2$ $\quad z_2 = w_x x_2 + w_a a_1 + b$

$\quad a_2 = f(z_2)$

a_2 vector captures the context of entire input sequence.

When we use bidirectional RNN/LSTM then we capture the final representation of input sequence as average of all tokens representation.



final sequence representation = $avg(a_0\ a_1\ a_2)$

Encoder: Captures the context and meaning of input sequence and stores them in vector representations

Decoder: uses the vector representations captured by encoder to generate relevant output sequence

Operations inside decoder:

vector representation from encoder : $a = avg(a_0\ a_1\ a_2)$

at $t=0$ in decoder

$$b_0 = W_x' x_0 + W_b' b_{-1} + b'$$

$x_0 \rightarrow$ [SOS] token $\rightarrow$ start of sentence token
it tells decoder to start generating the output sequence

at $t=0$ $b_{-1} = a \rightarrow$ vector repr. from encoder

$$b_0 = W'_x x_0 + W'_b a + b'$$

x_0 ↓ [sos] — a ↓ input from encoder

Decoder now needs to find the first token that it has to generate.

$b_0 \rightarrow$ vector

if we have 3 nodes in decoder RNN/LSTM

$$b_0 = [b_{00}\ b_{10}\ b_{20}]$$

We already have the hindi vocabulary stored in form of token and token id

lets say this vocabulary has 10k unique tokens

In language translation there will 2 vocabs

english vocab - for encoder

hindi vocab - for decoder

at t=0 using b_0 decoder has to find the token to start the output sequence

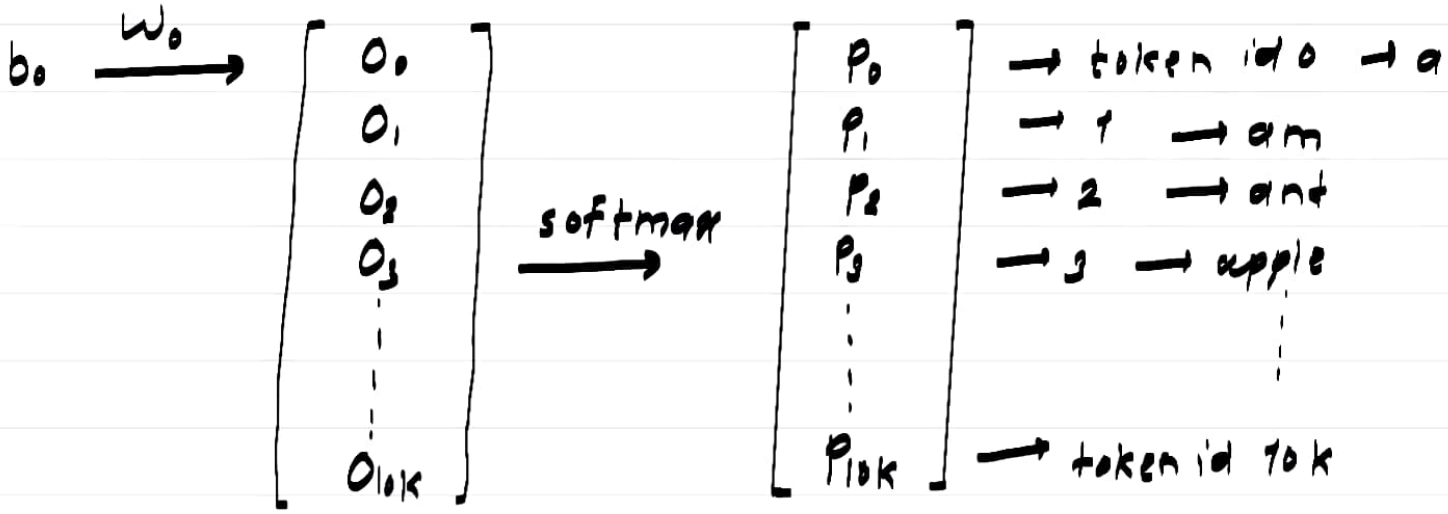
$$b_0 = \begin{bmatrix} b_{00} \\ b_{10} \\ b_{20} \end{bmatrix} \xrightarrow{W_0}$$



shape of W_0 = 3x10k

$W_0 \rightarrow$ output layer weight matrix

10k nodes - 1 node per token



$$p_i = \frac{e^{o_i}}{\Sigma e^{o}}$$

find maximum probability from p_0 to p_{10k}

the token with max probability is the first token that decoder generates

- output layer after decoder layer has 'n' nodes where n → decoder vocab size

- this output layer always has softmax activation

- output layer generates probability for every token in vocab

Loss computation:

at t=0 output layer generates first token as → 'ghar' ~~GR~~

↑ generated as weight are randomly initialized at start

but the correct first token is → 'Ramesh' / ~~राम~~

output layer has generated probability for every token
so we can get the probability it generated for token 'Ramesh'

lets say $p(\text{Ramesh}/\text{रमेश}) = P_{900}$

- as we are using softmax activation
- output layer is doing multiclass classification with each token being a class
- for multiclass clf we use cross entropy loss

at t=0

$$loss = -y_0 \log(P_{900})$$

y_0: 1 for Ramesh → probability of 'Ramesh'

$$loss = -\log(P_{900})$$

at t=1

$$loss = -y_1 \log(P_{207})$$

y_1: 1 for 'ganit' → prob. of 'ganit'

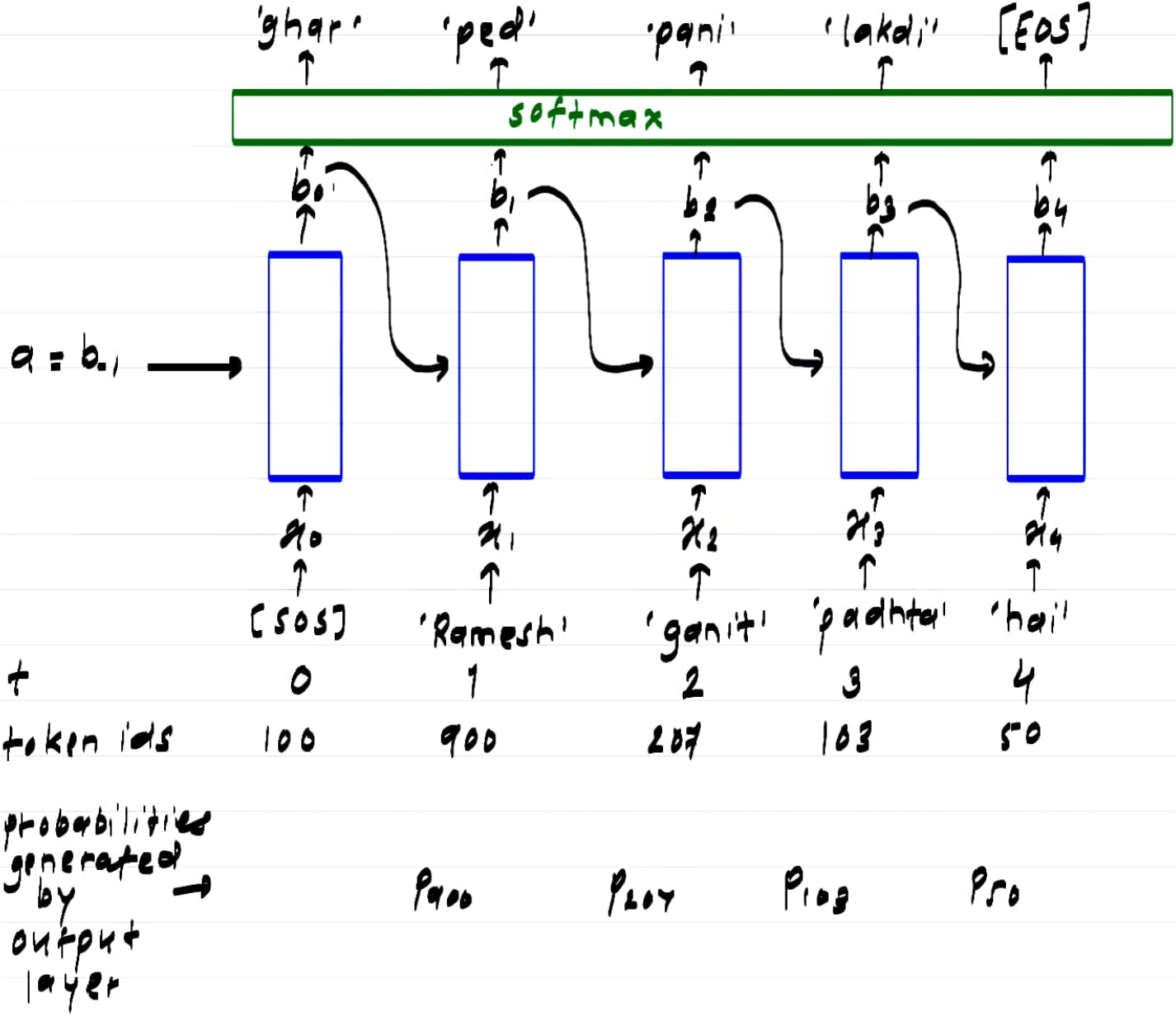
at t=1

decoder output

$$b_1 = w_h' x_1 + w_b' b_0 + b'$$

b_0: token id for 'Ramesh'

$b_1 \xrightarrow{W_0} \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_{10K} \end{bmatrix} \rightarrow P_{207}$



$$Loss = -[\log(p_{900}) + \log(p_{207}) + \log(p_{103}) + \log(p_{50})]$$

these probabilities are computed using

W_o → output layer weights

W'_a W'_b → decoder layer weights

W_a W_a → encoder layer weights

using chain rule, we can find

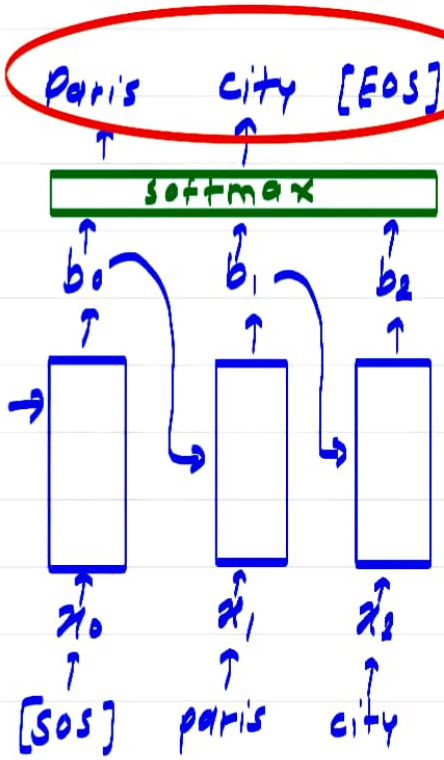
$\partial L/\partial W_o$ $\partial L/\partial W'_a$ $\partial L/\partial W'_b$ $\partial L/\partial W_a$ $\partial L/\partial W_a$

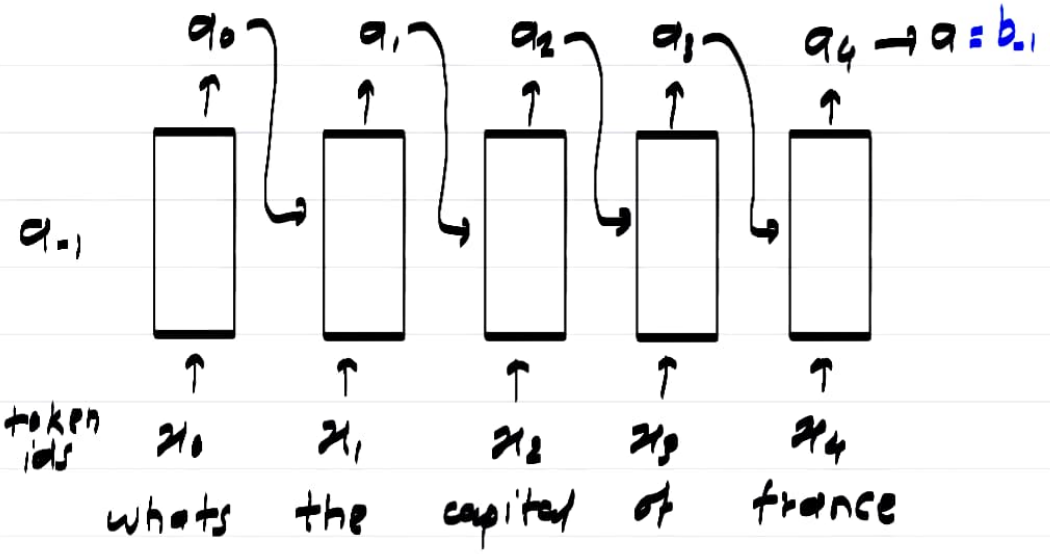
then update these weights to reduce loss

Que - Ans system :

Q : whats the capital of France ?

A : paris city





$$a = avg(a_0\ a_1\ a_2\ a_3\ a_4)$$